UAB
Universitat Autònoma de Barcelona

Fonaments de Xarxes / Xarxes I
Pràctica 1: Programació d'aplicacions amb *sockets*

Ramon Martí*, Gonçal Badenes*, Kevin Chow*, Joan Mata*, Vladimir Bellavista
{*ramon.marti.escale, goncal.badenes, kevin.chow, joan.mata*}@uab.cat
*Departament d'Enginyeria de la Informació i de les Comunicacions (DEIC)
Universitat Autònoma de Barcelona

Curs 2026-27

Índex

1 Introducció **1**

1.1 Objectius 1
1.2 Enunciat 1
1.3 Com treballem? 1
1.4 Implementació de la pràctica 2
1.4.1 VMWare i Màquina Virtual Debian 2
1.4.2 Instal·lació de WSL a Windows 3
1.4.3 Navegació pel terminal Linux 3
1.5 Funcionalitat 4
1.6 Funcionalitats del client 5
1.7 Funcionalitats del servidor 5
1.8 Concurrència 5
1.9 Com generar els executables: `client` i `server` 6
1.10 Execució del programa 6
1.11 Treball previ 6
1.12 Planificació de la pràctica per setmanes 7
1.13 Condicions de lliurament 8

Introducció

1.1 Objectius

En aquesta pràctica veureu les comunicacions entre dues aplicacions situades a màquines diferents utilitzant *sockets TCP*, orientats a connexió. Per això, desenvolupareu una parella d'aplicacions client/servidor en C (en el nostre cas, podem fer servir un sol ordinador i executar les dues aplicacions de manera simultània), especificant un petit protocol de 4 operacions per a la comunicació entre les aplicacions.

1.2 Enunciat

En aquesta pràctica, començarem amb la part de comunicació d'una aplicació de xat, que trobareu al Campus Virtual. L'objectiu és que un client pugui enviar missatges a un servidor i que aquest servidor respongui amb un missatge de confirmació. El client pot enviar múltiples missatges en una sessió fins que decideixi tancar la connexió. Recordeu que, en el nostre cas, el servidor i el client estan al mateix ordinador.

A partir d'aquesta aplicació simple de xat, cada equip haurà de definir un protocol de comunicació compost de 4 noves operacions (o més) per tal de donar algun tipus de servei simple.

Partireu de dos fitxers `server.c` i `client.c` els quals haureu de completar per aconseguir els requisits de l'enunciat. En cap cas es pot eliminar o modificar significativament el contingut original d'ambdós fitxers, a menys que sigui absolutament necessari, i només caldrà ampliar-ne la funcionalitat.

1.3 Com treballem?

Aquesta pràctica es pot fer amb els ordinadors del laboratori o amb qualsevol ordinador o màquina virtual que tingui un compilador de C.

Per accedir als ordinadors del laboratori, a la primera sessió de pràctiques el professorat us proporcionarà un nom d'usuari i *contrasenya* per a cada equip.

Els ordinadors dels laboratoris són accessibles i estan disponibles per realitzar connexions remotes a través del mecanisme de parella de claus pública - privada, juntament amb la connexió a través

de la Xarxa Privada Virtual (XPV) Fortinet, tant si la connexió es realitza des de dins com des de fora l'escola. Trobareu més informació al document "Accés remot als ordinadors dels laboratoris de la Part I " al Campus Virtual de l'assignatura.

També podeu utilitzar qualsevol ordinador o màquina virtual Linux (o Windows amb WSL) en la qual hi hagi instal·lat un compilador de llenguatge C (p. ex. `gcc`). Si teniu Linux i no el teniu instal·lat, podeu instal·lar `gcc` i, opcionalment, `make`:

```
sudo apt update && sudo apt upgrade
sudo apt install gcc make
```

1.4 Implementació de la pràctica

En el Campus Virtual trobareu un arxiu zip (`skel-xxxx.zip` (amb `xxxx` el curs actual), on hi ha el codi en llenguatge C d'un programa client (`client.c`) i un servidor (`server.c`, que permet enviar una cadena entre client i servidor, i és el codi que haureu d'utilitzar com a plantilla per a la implementació de les altres funcionalitats.

Veureu que, a més del codi, també hi ha comentaris on es demana que allà inclogueu l'explicació de les funcions i/o arguments, i també ho haureu de fer. **La prova de validació es basarà bàsicament en el fet que hagueu entès les operacions i/o els seus arguments i, per tant, aquestes respostes són tant o més importants que la funcionalitat o el codi de les operacions addicionals que implementeu.**

ATENCIÓ! L'opció recomanada és que utilitzeu el vostre ordinador. Si opteu per aquesta opció, cal que a la primera sessió ja vingueu amb l'ordinador o la màquina virtual corresponent, amb tot el necessari per compilar programes en llenguatge C ja instal·lat.

Des de la màquina virtual (i també des de Linux o WSL), podeu descarregar l'arxiu zip amb l'esquelet del programa `client.c` i `servidor.c` utilitzant el següent URL en un navegador:
`http://deic-home.uab.cat/~rmarti/fx/skel-2526.zip`

1.4.1 VMWare i Màquina Virtual Debian

En cas que en el vostre ordinador no pogueu executar el compilador `gcc` o no tingueu cap màquina virtual on fer-ho, també trobareu al Campus virtual un programari de virtualització VMWare (`VMware-workstation-17.6.1-24319023.exe` que utilitzarem a la segona pràctica), i una imatge d'una màquina debian (`maquinavirtual-debian12.zip`, que abans d'utilitzar haureu de descomprimir).

Qui feu servir la màquina virtual Debian que hi ha al Campus Virtual, potser heu anat a parar a la pantalla on només surt el fons de pantalla, però no cap icona. Per accedir al navegador o al

menú d'aplicacions, on trobareu el terminal, entre altres, aneu a l'opció "Actividades" a la part superior esquerra de la pantalla, i llavors a la part inferior ja veureu la icona del Firefox, i la icona d'un quadrat fet de 9 punts que és el menú d'aplicacions. Si cliqueu la icona del quadrat, ja us apareixerà el menú desplegable amb totes les aplicacions. De fet, aquesta icona ja apareix en arrencar, però la primera vegada us pot passar desapercebuda i podeu a anar a parar a la pantalla principal sense icona.

1.4.2 Instal·lació de WSL a Windows

Si teniu problemes amb la instal·lació de la màquina virtual a Windows, per a la pràctica 1 podeu instal·lar Windows Subsystem for Linux (WSL).

Obriu un terminal Windows (preferiblement Powershell) en mode administrador i poseu-hi

```
wsl -install
```

Després, reinicieu l'ordinador i ja tindreu instal·lada i a punt per utilitzar l'Ubuntu. També es poden triar altres distribucions de linux, com per exemple, debian, fedora, arch, kali, opensuse, o alma.

Podeu trobar la descripció més detallada a :
`https://learn.microsoft.com/en-us/windows/wsl/install`

Un cop dins la màquina virtual, convé actualitzar els paquets i instal·lar els que convinguin:

```
1 sudo apt update && sudo apt upgrade
2 sudo apt install gcc make
```

Llavors ja podreu utilitzar el `gcc` directament en un terminal Windows.

1.4.3 Navegació pel terminal Linux

Per veure el que hi ha dins un directori i moure's per ells, linux disposa de les ordres `ls` i `cd`, respectivament.

A continuació podeu trobar les ordres i els paràmetres que us seran de més utilitat per navegar pels directoris al terminal Linux:

- `cat` *`nom_fitxer`*: visualitza el contingut del fitxer *`nom_fitxer`*
- `cd`: canvia al directori home
- `cd ..` : canvia al directori pare
- `cd` *`directori`*: canvia al directori *`directori`*
- `cp` *`nom_fitxer_origen nom_fitxer_o_ruta_destinació`*: copia el fitxer *`nom_fitxer_origen`* com a fitxer *`nom_fitxer_destinació`* o dins el directori *`nom_ruta_destinació`*. Per copiar

a la carpeta actual, s'ha d'utilitzar un punt (.) com a *ruta_destinació*.

- `ls`: llista el directori actual
- `ls -l`: llista + informació del directori actual
- `ls -la`: llista + informació + fitxers ocults del directori actual
- `ls \camí`: llista el directori `\camí`
- `mkdir directori`: crea el directori *directori*
- `passwd`: canvia la contrasenya d'un compte d'usuari
- `pwd`: imprimeix el directori actual (de treball) (print working directory)
- `rm nom_fitxer`: elimina (suprimeix) el fitxer *nom_fitxer*

1.5 Funcionalitat

L'aplicació client mostrarà a l'usuari un menú per poder escollir una de les opcions possibles, entre les quals hi haurà:

- **Enviar missatge:** L'usuari escriurà un missatge que s'enviarà al servidor. El servidor rebrà el missatge, el mostrarà per pantalla i enviarà una resposta de confirmació al client amb el text que el client ha enviat, afegint-hi "`.  Missatge rebut`". Per exemple, si s'ha enviat la frase "`Hola, món`", el servidor ha de respondre "`Hola, món.  Missatge rebut`".

  IMPORTANT: Vegeu que l'esquelet que us donem retorna exactament la mateixa cadena rebuda, i per tant heu de modificar lleugerament el servidor per tal que retorni la cadena que us demanem.

- **Sortir:** Tanca la connexió amb el servidor. El client envia un missatge especial "`EXIT`" per indicar al servidor que ha de tancar la connexió.

- **4 Funcionalitats (operacions) extra:** Heu de primer especificar el protocol i posteriorment implementar almenys 4 funcionalitats noves que implementin algun tipus de servei remot, amb comunicació entre el servidor i el client. Han d'oferir algun servei concret (les 4 operacions han de seguir un cert fil), han de ser senzilles, i que sapigueu explicar la seva implementació de la part de sockets.

  Cal que **definiu vosaltres mateixos el protocol** (i el porteu a l'inici de la 1a sessió de laboratori): sintaxi (significat), semàntica (codificació) i ordre (si cal) de les operacions i les dades a demanar, enviar (i analitzar al servidor) i rebre. Si per donar més sentit al servei cal alguna opció més, la podeu afegir.

Nota: L'avaluació no dependrà de la dificultat en la implementació de la funcionalitat en el servidor. Sí que es penalitzarà que els missatges entre client i servidor no segueixin cap fil (no

tinguin relació entre ells), o que siguin molt-molt simples (p. ex. només el número escollit al menú per identificar una operació sense arguments).

1.6 Funcionalitats del client

1. En executar-se, el client mostrarà un menú amb les opcions disponibles.
2. Permetrà a l'usuari introduir els arguments de l'operació corresponent.
3. Generarà un missatge amb l'identificador d'operació i el(s) argument(s), i l'enviarà al servidor.
4. Rebrà la resposta del servidor al missatge enviat i la mostrarà per pantalla.
5. Quan l'usuari seleccioni l'opció de sortir, el client enviarà el missatge "`EXIT`" per tancar la connexió amb el servidor.

1.7 Funcionalitats del servidor

1. En executar-se, el servidor es posarà en espera de rebre connexions de clients.
2. En rebre una connexió, estarà preparat per rebre missatges amb les operacions del client.
3. Quan rebi un missatge corresponent a l'opció d'enviar missatge (1), mostrarà el contingut per pantalla i enviarà al client com a resposta una cadena que inclogui el missatge rebut més "`. Missatge rebut`".
4. Quan rebi el missatge "`EXIT`", el servidor tancarà la connexió amb el client de manera segura.
5. Quan rebi qualsevol altre missatge (corresponent a les vostres funcionalitats/operacions extra), llegirà l'identificador d'operació i els arguments, executarà l'operació corresponent i retornarà el resultat al client.

1.8 Concurrència

Quan el servidor rep una connexió d'un client, passa a executar les ordres que el client li demana. Si mentrestant altres clients s'hi volen connectar, els encua fins que no hagi acabat amb l'ordre que està executant actualment. Tenim doncs un servidor iteratiu.

Si volem que més d'un client es pugui connectar, necessitarem preparar el servidor per rebre múltiples connexions de diferents clients. Caldrà, doncs, que modifiqueu el servidor perquè sigui concurrent. D'aquesta manera, el servidor podrà atendre a la vegada totes les diferents peticions de diferents clients.

Per fer-ho cal que:

1. El servidor tingui un **socket** permanentment escoltant per un port possibles connexions entrants.
2. En rebre una connexió, el servidor ha de crear un altre **socket**, associat a un altre port diferent per atendre les possibles operacions que ens enviï el client i enviar-li les respostes. Quan el client surti de l'aplicació, el servidor tancarà aquest **socket**.

1.9 Com generar els executables: `client` i `server`

Per generar els fitxers que es puguin executar: `client` i `server`, cal que compilem el codi i generem els executables.

Des d'un terminal, en el directori on tinguem el codi font del client i del servidor, utilitzarem les següents ordres per compilar-los:

```
gcc -o client client.c
gcc -o server server.c
```

1.10 Execució del programa

Per executar el programa, caldrà primer obrir 2 terminals, un per al client i l'altre per al servidor, en el mateix directori on hi ha els fitxers compilats (que és el mateix on hi ha els fitxers en codi font).

Primer caldrà llençar l'aplicació `server` en un dels terminals, i la sintaxi serà la següent:

`./server` *`número_port`*

Per a número de port, utilitzeu el 50000 més els 4 últims dígits del NIU d'1 dels membres de l'equip. P. ex. si el NIU és 1234567, el port seria el `54567`.

I després, en l'altre terminal, ja es podrà llençar l'aplicació `client` amb la següent sintaxi:

`./client` *`nom_o_ip_servidor número_port`*

Si el servidor està al mateix equip que el client (cas d'aquesta pràctica), utilitzeu **`localhost`** com a primer argument.

Tingueu en compte que haureu d'executar primer el servidor i després el client, i que cadascun s'haurà de cridar en un terminal diferent (un terminal per al servidor i un altre per al client).

1.11 Treball previ

A l'inici de la primera sessió de laboratori, caldrà que cada grup tingui el sistema configurat i el protocol a implementar ja especificat:

- Si voleu treballar amb el vostre ordinador portàtil (opció recomanada), cal portar el sistema configurat de manera que es pugui executar el compilador `gcc`, ja sigui amb un ordinador

Linux, una Màquina Virtual amb Linux, Windows Subsystem for Linux (WSL), macOS...).

- Cal també haver descarregat l'esquelet de la pràctica del Campus Virtual, descomprimit, copiat al sistema, i compilat, i caldrà mostrar al professorat que es compila i s'executa correctament.
- També caldrà que tots els grups **portin imprès o escrit en paper, on s'inclogui a la part superior el nom i cognom dels 2 membres de l'equip**, així com l'especificació del protocol associat a les quatre (o més) operacions que hagin decidit implementar:
  - **Semàntica**: L'explicació de les operacions a implementar, incloent-hi el(s) argument(s) i resultat(s) i els seus tipus de dada corresponents.
  - **Sintaxi**: Com es codificaran les operacions (l'identificador de cada operació), arguments i resultats per a la seva transmissió entre client i servidor. Normalment, ho farem en format text, però a més de l'identificador de cada operació, també caldrà definir com se separarà aquest identificador dels arguments, i els arguments entre ells (espai en blanc, coma, punt, punt i coma, combinació, etc.).
  - **Ordre**: Si en el protocol especificat cal que algunes o totes les operacions segueixin un ordre concret, caldrà especificar-ho. Si no cal, també caldrà especificar que no segueixen cap ordre.

1.12 Planificació de la pràctica per setmanes

Inici de la 1a sessió ([FX: 29/09; 01,02,06,08,09/10] [X1: 21,22,28,29/09]): Portar el sistema configurat i l'especificació del protocol escrita o impresa en paper, on hi hagi en la part superior el nom i cognom dels 2 membres de l'equip.

1a sessió ([FX: 29/09; 01,02,06,08,09/10] [X1: 21,22,28,29/09]): Implementació de la configuració dels sockets i la funcionalitat bàsica d'enviament i recepció de missatges.

Entre la 1a sessió i la 2a sessió: Implementació al màxim possible de la funcionalitat de les operacions, incloent-hi la implementació de les connexions concurrents.

2a sessió ([FX: 13,15,16,20,22,23/10] [X1: 05,06,13,19/10]): Prova, depuració i presentació al professorat del sistema, incloent-hi les connexions concurrents.

Durant la 2a sessió, es revisaran les funcionalitats addicionals. Per tant, heu de portar la pràctica realitzada o tan avançada com sigui possible per tal de ser avaluada, ja que l'avaluació d'aquesta part es farà "in situ".

Prova de validació de la Pràctica 1 (28/10, 15:00): La prova de validació de la pràctica 1 es realitzarà el mateix dia que l'examen parcial de la Part 1. Alguns torns de pràctiques no hauran fet la 2a sessió de pràctiques, però la prova de validació serà de conceptes generals de

l'enunciat i de sockets, incloent-hi les connexions concurrents.

1.13 Condicions de lliurament

Llegiu-vos atentament l'enunciat abans del lliurament i verifiqueu que la vostra pràctica té implementat tot el que es demana.

El lliurament de la pràctica tindrà un mateix dia límit per a tots els grups, serà a través del Campus Virtual.

- FX: el diumenge **25/10/26, 23:59**
- X1: el diumenge **18/10/26, 23:59**

Comprimiu en un arxiu (zip o rar):

- Els fitxers `client.c` i `server.c`. Recordeu que, a més de la funcionalitat demanada, també han d'incloure, on es demana, els comentaris explicant les funcions i/o els arguments.
- Un document pdf amb l'especificació del protocol presentat en paper a l'inici de la 1a sessió de laboratori.

IMPORTANT:

- El nom del fitxer comprimit ha de començar per l'identificador de l'equip (p. ex. `fxt-a3` o x1-a3), seguit de "`-Pràctica1`". Opcionalment, a continuació, i separat també per un guió, podeu posar altra informació com els vostres cognoms, o el que cregueu oportú. Exemples: `fxt-a3-Pràctica1.zip`, `x1-c4-Pràctica1-Martí-Mata.zip`, ...
- Només 1 dels 2 membres de l'equip ha de fer el lliurament.
- Inclogueu també, com a comentari, l'**identificador de l'equip i el nom i cognom** dels membres a la **part superior de cada fitxer**.
- **Qualsevol pràctica que no inclogui l'identificador d'equip i el número de pràctica a l'inici del nom del fitxer lliurat NO SERÀ AVALUADA.**